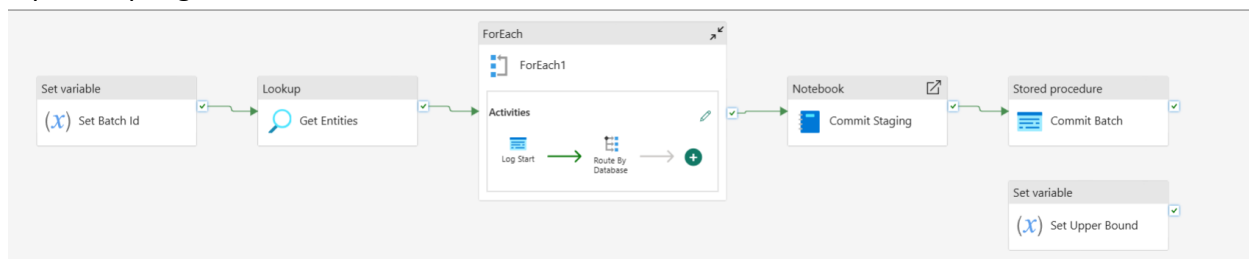


Healthcare Analytics Lakehouse

- Started with data generation with the help of AI. Not able to find a reliable dataset and source.
- Step 0 includes data generation using python and split the data into five set. Then using these five set as different data source system.
- Source-system overview

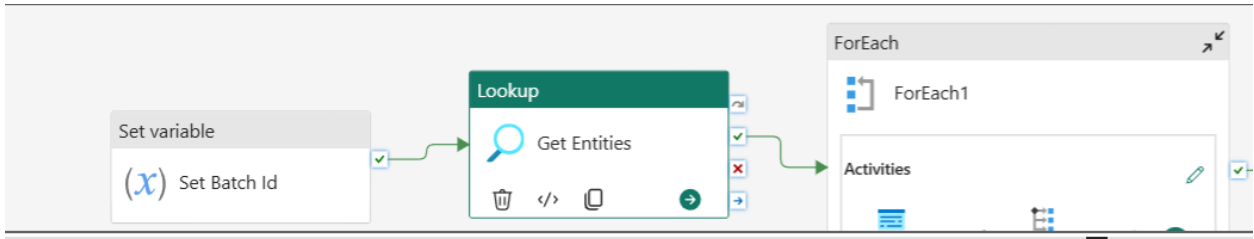
Source	System	Entities	Interface	Load type
EHR	Hospital EHR / ADT	patient, encounter, admission, discharge, bed_assignment, diagnosis, procedure	SQL Server	Incremental
SCHED	Appointment scheduling	appointment, appointment_status_history, clinic_slot	Azure SQL + change tracking	Incremental
LIS	Laboratory information system	lab_order, lab_result, specimen	HL7 v2 / flat file	Incremental
PHARM	Pharmacy	medication_order, dispense, formulary	REST API	Incremental
CLAIMS	Insurance platform	claim_header, claim_line, adjudication, payer	EDI 837/835 over SFTP	Daily
FIN	Billing / ERP	invoice, invoice_line, payment, adjustment	Oracle	Daily
HR	Staffing	doctor, staff, department_assignment, credential	CSV export	Daily snapshot
SURVEY	Patient satisfaction	survey_response, survey_question	SaaS API	Daily
FACIL	Facility management	hospital, department, ward, room, bed	CSV export	Daily snapshot
REF	Reference vocabularies	ICD-10-CA, LOINC, DIN/ATC, CCI	Manual upload	Quarterly

- Created 4 lakehouse [LH_Bronze, LH_Silver, LH_Gold, LH_Governance]
- 1 warehouse
- Step 1: Data Ingestion | Source SQLServer
DBO: EHR | SCHED | FIN
Pipeline: plIngestSQL



- Set Batch ID (setVariable) for assigning batchID for a particular run and appended as a column in the dataset.
- Set Upper Bound (setVariable) for assing watermark value to utcNow .

- Get Entities (Lookup) using stored procedure to get entities(table names) and pass on to foreach.



The screenshot shows a workflow editor with three main activities: 'Set variable' (containing 'Set Batch Id'), 'Lookup' (containing 'Get Entities'), and 'ForEach' (containing 'ForEach1'). The 'Lookup' activity is highlighted, and its settings are shown in the panel below.

Settings

Connection * WH-OHN-CONTROL (OHN-dev) Refresh Open

Use query ☐ Table ☐ Query ☒ Stored procedure

Stored procedure name * [ctl].[sp_get_entities_to_ingest] Refresh Preview data

Stored procedure parameters ⓘ

Import parameter

Parameter + New Delete

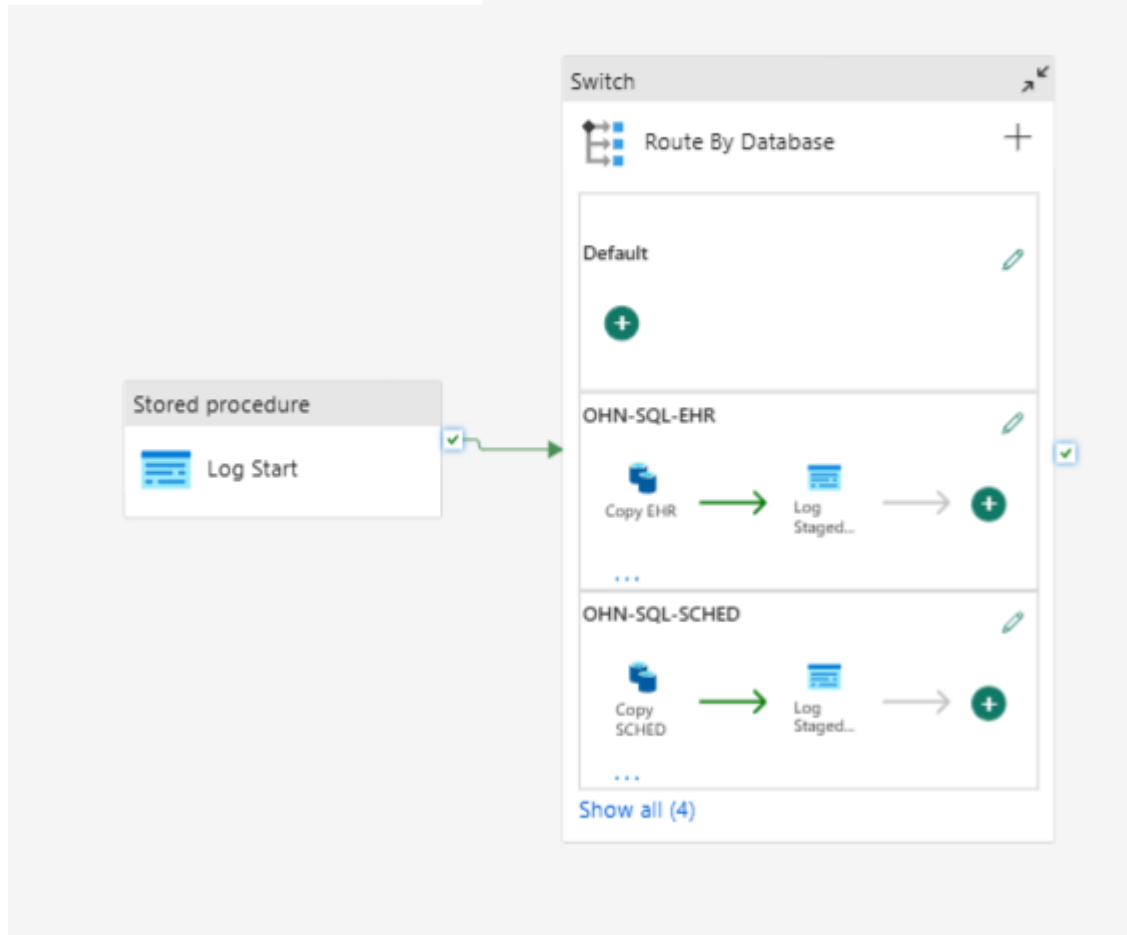
☐	Name	Type *	Value	☐	Treat as null
☐	connection_type	String	sqlserver_gateway	☐	Treat as null

First row only ☐

> Advanced

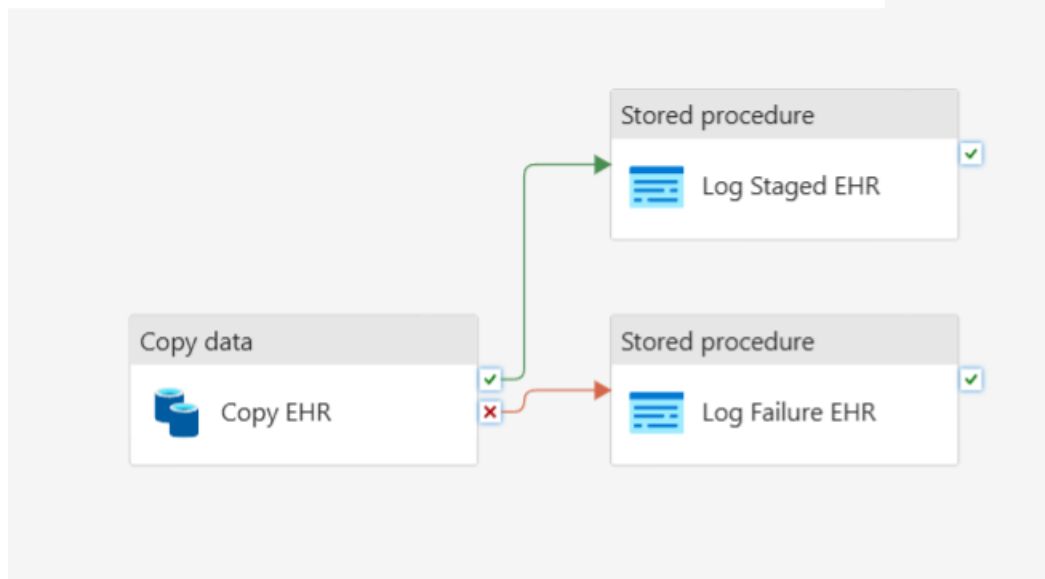
- This will get 11 entities
 - o EHR: Admission | Bed_Assignment | Diagnosis | Emergency_Visist | Patient
 - o FIN: Invoice | Invoice_line | Patient_Account
 - o Sched: Apooointment | appointment_status_history | Patient
- ForEach:

Main canvas >  ForEach1



○

Main canvas >  ForEach1 >  Route By Database - OHNSQLEHR



○

LogStart: Assign parameters and variables to start with data ingestion.

General

Settings

Connection *

WH-OHN-CONTROL (OHN-dev)

Refresh

Open

Stored procedure name *

[ctl].[sp_batch_start]

Refresh

Stored procedure parameters ⓘ

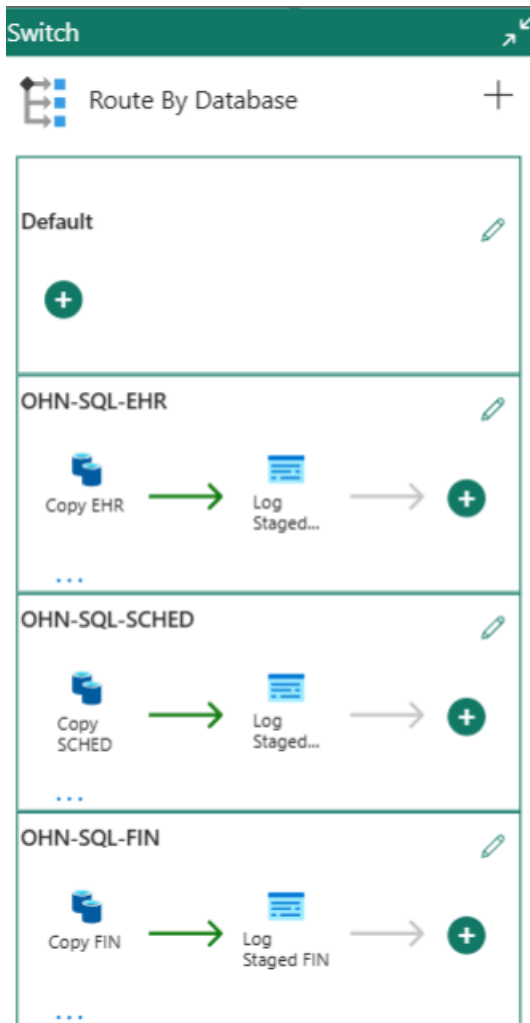
Import

New

Delete

☐	Name	Type *	Value
☐	batch_id	String	@variables('batch_id')
☐	entity_name	String	@item().entity_name
☐	load_type	String	@item().load_type
☐	pipeline_name	String	@pipeline().Pipeline
☐	pipeline_run_id	String	@pipeline().RunId
☐	source_system	String	@item().source_system
☐	target_table	String	@item().target_table
☐	watermark_from	String	@item().watermark_value
☐	watermark_to	String	@variables('load_upper_bound')

SwitchCase: Runs over the 3 databases



Inside 1 case: OHN-SL-HER

CopyActivity: getting data from Onprem SQL Server to LH_Bronze

The screenshot displays the Microsoft Fabric Data Gateway configuration interface. The top section shows a visual workflow with a 'Copy data' task connected to two 'Stored procedure' tasks: 'Log Staged EHR' and 'Log Failure EHR'. The bottom section shows the configuration for the 'Copy data' task, including connection details and query options.

General **Source** Destination Mapping Settings

Connection * OHN-SQL-EHR Refresh Edit

Connection type SQL Server Test connection

Database OHN_EHR Refresh

Use query ☐ Table ☒ Query ☐ Stored procedure

Query @concat('SELECT * FROM ', item().so... Preview data

> Advanced

For this ingestion, I have been using Docker to setup SQLserver image to transfer data over localhost:1433. Using 'On-Prem Datagateway', I setup a gateway connection between localhost and my Fabric User account. One new connection has been assigned inside Fabric 'Managed Connectiontions and Gateways'.

Manage Connections and Gateways

Connections

[On-premises data gateways](#)[Virtual network data gateways](#)[Streaming virtual network data gateways](#)[Azure Key Vault references](#)

Cloud and data gateway connections for artifacts. [Learn more about supported connections.](#)

Name ↑	Connection type	Users	Status	Gateway cluster name	Last credentials used ⓘ
CopyJob user1	Copy job	User 1	🟢		8/3/26, 10:17 PM
https://wx56c3j3-8000.use.devunne...	Web v2	User 1	🟢	OHN-gateway-local	8/9/26, 11:07 PM
OHN-API	Web	User 1	🟢	OHN-gateway-local	8/9/26, 9:58 PM
OHN-SQL-EHR	SQL Server	User 1	🟢	OHN-gateway-local	8/11/26, 5:02 PM
OHN-SQL-FIN	SQL Server	User 1	🟢	OHN-gateway-local	8/11/26, 5:04 PM
OHN-SQL-SCHED	SQL Server	User 1	🟢	OHN-gateway-local	8/4/26, 5:58 AM

Query: @concat(

```
'SELECT * FROM ', item().source_object,  
if(equals(item().load_type, 'incremental'),  
concat(' WHERE ', item().watermark_column,  
      '> ', item().watermark_value, ''',  
      ' AND ', item().watermark_column,  
      '<= ', variables('load_upper_bound'), '''),  
''')  
)
```

General	Source	Destination	Mapping	Settings
Connection * lh_bronze (OHN-dev) Refresh Open				
Root folder ☒ Tables ☐ Files				
Table * schema name . @concat('stg_', item().target_table) Preview data + New				
☒ Enter manually				
Table action ☐ Append ⓘ ☒ Overwrite ⓘ ☐ Upsert ⓘ				
> Advanced				

Source incoming data will be saved temporary named with prefix 'STG' with overwrite action. Since I am gonna push all these staged table at last with the help of notebook, so overwrite action is being used for these intermediary tables.

If Copy success, then it will be logges as success.

Main canvas > ForEach1 > Route By Database - OHNSQLEHR



General **Settings**

Connection *

WH-OHN-CONTROL (OHN-dev)

Refresh

Open

Stored procedure name *

[ctl].[sp_batch_end]

Refresh

Stored procedure parameters ⓘ

Import

+

New

🗑

Delete

☐	Name	Type *	Value
☐	batch_id	String	@variables('batch_id')
☐	entity_name	String	@item().entity_name
☐	error_message	String	Value ☒ Treat as null
☐	rows_read	Int64	@int(activity('Copy EHR').output.row...)
☐	rows_written	Int64	@int(activity('Copy EHR').output.row...)
☐	source_system	String	@item().source_system
☐	status	String	STAGED ☐ Treat as null

If fails

Main canvas > ForEach1 > Route By Database - OHNSQLEHR

Copy data

Copy EHR

Stored procedure

Log Failure EHR

General **Settings**

Connection * WH-OHN-CONTROL (OHN-dev) Refresh Open

Stored procedure name * [ctl].[sp_batch_end] Refresh

✓ Stored procedure parameters ⓘ

↔ Import + New Delete

☐	Name	Type *	Value
☐	batch_id	String	@variables('batch_id')
☐	entity_name	String	@item().entity_name
☐	error_message	String	@activity('Copy EHR').output.errors[...]
☐	rows_read	Int64	0 ☐ Treat as null
☐	rows_written	Int64	0 ☐ Treat as null
☐	source_system	String	@item().source_system
☐	status	String	FAILED ☐ Treat as null

After success of forEach, we now have 11 staged tables ready to commit into bronze layer with additional appended columns with (batchID, watermark, etc)

Next -> Notebook Activity: Commit Staging with two base parameters batchID and ENTITIES.

Try the new pipeline zoom feature for better visibility on large pipelines!

General **Settings**

Please review this item carefully before adding it to the pipeline, as others in your organization may have access to notebooks in this workspace.

Connection Select... Refresh

Workspace * OHN-dev Refresh

Notebook * 13_bronze_commit_staging Refresh Open + New

✓ Base parameters

+ New | Delete

☐	Name	Type *	Value
☐	batch_id	String	@variables('batch_id')
☐	entities	String	@string(activity('Get Entities').output...

> Advanced settings

> Spark settings

Notebook Snapshots:

```
1 # This cell is generated from runtime parameters. Learn more: https://go.microsoft.com/fwlink/?linkid=2161015
2 batch_id = "SQLSRV_20260804101956_f1ec9b4e"
3 entities = "[{'source_system':'EHR','entity_name':'admission','connection_type':'sqlserver_gateway','connection_name':'OHN-SQL-EHR','so
4
```

[1] ✓ - Command executed in 300 ms on 8/4/2026, 6:23:37 AM

```
15 # All table names are UNQUALIFIED and resolve against the attached default
16 # lakehouse. Hardcoding a "lh_bronze." prefix ties the notebook to one
17 # lakehouse name; when the item was actually called h1_bronze, every
18 # spark.table() call raised AnalysisException, the loop swallowed it as
19 # "no staging table", and eleven entities silently committed nothing.
20 existing = {t.tableName for t in spark.sql("SHOW TABLES").collect()}
21 print(f"Committing batch {batch_id} - {len(entity_list)} entities")
22 print(f"Tables visible in default lakehouse: {len(existing)}")
23 print(f"Staging tables found: {sorted(t for t in existing if t.startswith('stg_'))}\n")
```

[4] ✓ - Command executed in 7 sec 275 ms on 8/4/2026, 6:23:45 AM

Committing batch SQLSRV_20260804101956_f1ec9b4e - 11 entities
Tables visible in default lakehouse: 11
Staging tables found: ['stg_ehr_admission', 'stg_ehr_bed_assignment', 'stg_ehr_diagnosis', 'stg_ehr_emergency_visit', 'stg_ehr_patient', 'stg_fin_invoice', 'stg_fin_invoice_line', 'stg_fin_patient_account', 'stg_sched_appointment', 'stg_sched_appointment_status_history', 'stg_sched_patient']

```

64     spark.sql(f"DROP TABLE IF EXISTS {staging_table}")
65
66     print(f"  {label:<34} {row_count:>9,} rows  ({mode_used})")
67     results.append({"entity": label, "target": target_table,
68                   "rows": row_count, "status": "COMMITTED"})

```

[5] ✓ - Command executed in 1 min 46 sec 288 ms on 8/4/2026, 6:25:31 AM

EHR.admission	11,568 rows	(append)
EHR.bed_assignment	14,804 rows	(append)
EHR.diagnosis	37,318 rows	(append)
EHR.emergency_visit	25,037 rows	(append)
EHR.patient	25,998 rows	(append)
FIN.invoice	11,568 rows	(append)
FIN.invoice_line	52,196 rows	(append)
FIN.patient_account	15,985 rows	(append)
SCHED.appointment	228,833 rows	(append)
SCHED.appointment_status_history	457,666 rows	(append)
SCHED.patient	20,448 rows	(append)

```

32     mssparkutils.notebook.exit(json.dumps({
33         "batch_id": batch_id,
34         "entities_committed": len(committed),
35         "total_rows": total_rows,
36     }))

```

[6] ✓ - Command executed in 24 sec 234 ms on 8/4/2026, 6:25:56 AM

Committed : 11 entities, 901,421 rows

No data : 0 entities

Reconciliation: staging and Bronze row counts match for every entity

ExitValue: {"batch_id": "SQLSRV_20260804101956_f1ec9b4e", "entities_committed": 11, "total_rows": 901421}

At Last, pipeline ended with Commit Staged SP

The screenshot shows the Azure Data Factory pipeline editor. The pipeline consists of a **ForEach** loop containing a **Log Start** activity and a **Route By** activity. The **Route By** activity is connected to a **Notebook** activity named **Commit Staging**, which is then connected to a **Stored procedure** activity named **Commit Batch**. The **Stored procedure** activity is configured with the connection **WH-OHN-CONTROL (OHN-dev)** and the stored procedure name **[ctl].[sp_commit_batch]**. The **Stored procedure parameters** section shows a parameter **batch_id** of type **String** with the value **@variables('batch_id')**.

General **Settings**

Connection * **WH-OHN-CONTROL (OHN-dev)** Refresh Open

Stored procedure name * **[ctl].[sp_commit_batch]** Refresh

Stored procedure parameters ⓘ

Import + New Delete

☐	Name	Type *	Value
☐	batch_id	String	@variables('batch_id')

> Advanced

Step 2: Data Ingestion over API

For this step I have transmitted data over localhost8000 and againnn with the 'Managed connections and Gateways' in fabric and On-Premise Gateway on my local system, connection has been set up.

Starts with Pipeine: plIngestAPI

Try the new pipeline zoom feature for better visibility on large pipelines!

General Settings

Connection * Refresh Edit

Relative URL

Method *

Body

Headers *

Name	Value
☐ Content-Type	application/x-www-form-urlencoded

> Advanced

First is to get authorization token, then storing that token with the help of setVariable - STORE TOKEN.

Try the new pipeline zoom feature for better visibility on large pipelines!

General Settings

Variable type ☒ Pipeline variable ☐ Pipeline return value

Name *

Value

GetAPIEntities: lookup for entities and over connection_type REST API, it will count to four as follows:

```
{
  "count": 4,
  "value": [
    {
      "source_system": "FACIL",
      "entity_name": "bed",
      "connection_type": "rest_api",
      "connection_name": "OHN-API",
      "source_object": "beds",
      "target_table": "facil_bed",
    }
  ]
}
```

```

        "business_key": "bed_id",
        "watermark_column": "update_ts",
        "load_type": "full_snapshot",
        "file_format": "json",
        "sla_minutes": 1440,
        "watermark_value": "1900-01-01 00:00:00"
    },
    {
        "source_system": "FACIL",
        "entity_name": "department",
        "connection_type": "rest_api",
        "connection_name": "OHN-API",
        "source_object": "departments",
        "target_table": "facil_department",
        "business_key": "department_id",
        "watermark_column": "update_ts",
        "load_type": "full_snapshot",
        "file_format": "json",
        "sla_minutes": 1440,
        "watermark_value": "1900-01-01 00:00:00"
    },
    {
        "source_system": "FACIL",
        "entity_name": "hospital",
        "connection_type": "rest_api",
        "connection_name": "OHN-API",
        "source_object": "hospitals",
        "target_table": "facil_hospital",
        "business_key": "hospital_id",
        "watermark_column": "update_ts",
        "load_type": "full_snapshot",
        "file_format": "json",
        "sla_minutes": 1440,
        "watermark_value": "1900-01-01 00:00:00"
    },
    {
        "source_system": "PHARM",
        "entity_name": "medication_order",
        "connection_type": "rest_api",
        "connection_name": "OHN-API",
        "source_object": "medication-orders",
        "target_table": "pharm_medication_order",
        "business_key": "medication_order_id",
        "watermark_column": "updated_at",
        "load_type": "incremental",
        "file_format": "json",
        "sla_minutes": 60,
        "watermark_value": "1900-01-01 00:00:00"
    }
}

```

FOREACH Activity with count=4 iterate over each entity and transfer data with the help of copy activity

HomeActivitiesRunView

Validate

Run

Schedule

Trigger

View run history

Copy data

Try the new pipeline zoom feature for better visibility on large pipelines!

Main canvas > For Each API Entity

Stored procedure

Log Start

Copy data

Copy API

Stored procedure

Log Staged

Stored procedure

Log Failure

GeneralSourceDestinationMappingSettings

Connection *

https://wx56c3j3-8000.use.devu...

RefreshTest connectionEdit

Additional connection properties

Relative URL

@concat('/api/v1/', item().source_obj...

Preview data

Advanced

Pipeline expression builder

Add dynamic content below using any combination of expressions, functions and system variables.

```
@concat('/api/v1/', item().source_object,
'?limit=1000',
if(equals(item().load_type,'incremental'),
concat('&updated_since=', replace(item().watermark_value,
',', 'T')),
''))
```

Clear contentsEvaluate expression

ForEach iterator

Activity outputs

Parameters

System variables

...

Search

For Each API Entity

Current item

OK

Cancel

Applies pagination rules to QueryParamenter.cursor

GeneralSourceDestinationMappingSettings

Request interval (ms)

10

Additional headers

+ New

Delete

Value

Authorization

@(concat('Bearer ', variables('access_...

Pagination rules

+ New

Delete

Name

Value

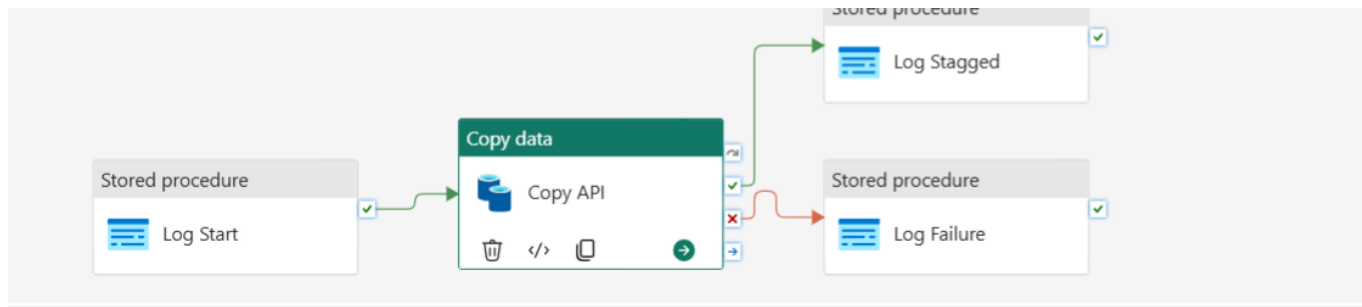
QueryParame...

cursor

Body

pagination.next_cursor

Same as last, the tables will be destined temporary as staged tables with overwrite table action.



General Source **Destination** Mapping Settings

Connection * lh_bronze (OHN-dev) Refresh Open

Root folder ☒ Tables ☐ Files

Table * schema name . @concat('stg_', item().target_table) Preview data New

☒ Enter manually

Table action ☐ Append ⓘ ☒ Overwrite ⓘ ☐ Upsert ⓘ

> Advanced

For this pipeline, data mapping has also been applied

General Source Destination **Mapping** Settings

← Import schemas Preview source + New mapping Reset ⓘ Delete

☐	Source	Type		Destination	Type		
☐	data.bed_id	abc String	→	bed_id	abc string	+	🗑
☐	data.room_number	abc String	→	room_number	abc string	+	🗑
☐	data.ward_name	abc String	→	ward_name	abc string	+	🗑
☐	data.ward_type	abc String	→	ward_type	abc string	+	🗑
☐	data.bed_type	abc String	→	bed_type	abc string	+	🗑
☐	data.hospital_id	abc String	→	hospital_id	abc string	+	🗑
☐	data.department_id	abc String	→	department_id	abc string	+	🗑
☐	data.is_isolation_capa...	abc String	→	is_isolation_capable	abc string	+	🗑
☐	data.is_active	abc String	→	is_active	abc string	+	🗑
☐	data.update_ts	abc String	→	update_ts	abc string	+	🗑
☐	pagination.returned	12l Int64	→	returned	12l long	+	🗑
☐	pagination.total_matc...	12l Int64	→	total_matching	12l long	+	🗑
☐	pagination.limit	12l Int64	→	limit	12l long	+	🗑
☐	pagination.has_more	Boolean	→	has_more	boolean	+	🗑

After running copy activity, it will be LOGGED as SUCCESS or Failure depending upon the run.

Main canvas > For Each API Entity

Stored procedure Log Start

Copy data Copy API

Stored procedure Log Stagged

Stored procedure Log Failure

General Settings

Connection * WH-OHN-CONTROL (OHN-dev) Refresh Open

Stored procedure name * [ctl].[sp_batch_end] Refresh

Stored procedure parameters ⓘ

Import + New Delete

☐	Name	Type *	Value
☐	batch_id	String	@variables('batch_id')
☐	entity_name	String	@item().entity_name
☐	error_message	String	Value ☒ Treat as null
☐	rows_read	Int64	@int(activity('Copy API').output.rows...

Then All the 4 staged entities committed to bronze lakehouse with the help of same notebook used previously for SQL ingestion with base parameters BatchOD and Entity Names.

ForEach

For Each API Entity

Activities

Log Start → Copy API → Log Staged → +

Notebook

Commit Staging

Stored procedure

Commit Batch

Set variable

General Settings

Connection ⓘ Select... Refresh

Workspace * OHN-dev Refresh

Notebook * 13_bronze_commit_staging Refresh Open + New

✓ Base parameters

+ New | Delete

☐	Name	Type *	Value
☐	batch_id	String	@variables('batch_id')
☐	entities	String	@string(activity('Get API Entities').ou...

> Advanced settings

> Spark settings

Notebook Snapshots:

```

1  # This cell is generated from runtime parameters. Learn more: https://go.microsoft.com/fwlink/?linkid=2161015
2  batch_id = "API_20260810030756_e96a7258"
3  entities = "[{"source_system":"FACIL","entity_name":"bed","connection_type":"rest_api","connection_name":"OHN-API","source_object":"b
4

```

1] ✓ - Command executed in 294 ms on 8/9/2026, 11:12:38 PM

```

19  # "no staging table", and eleven entities silently committed nothing.
20  existing = {t.tableName for t in spark.sql("SHOW TABLES").collect()}
21  print(f"Committing batch {batch_id} - {len(entity_list)} entities")
22  print(f"Tables visible in default lakehouse: {len(existing)}")
23  print(f"Staging tables found: {sorted(t for t in existing if t.startswith('stg_'))}\n")

```

[4] ✓ - Command executed in 5 sec 398 ms on 8/9/2026, 11:12:44 PM

Committing batch API_20260810030756_e96a7258 - 4 entities

Tables visible in default lakehouse: 15

Staging tables found: ['stg_facil_bed', 'stg_facil_department', 'stg_facil_hospital', 'stg_pharm_medication_order']

```

58         .mode("append")
59         .option("mergeSchema", "true")
60         .partitionBy("_load_date")
61         .saveAsTable(bronze_table))
62     mode_used = "append"
63
64     spark.sql(f"DROP TABLE IF EXISTS {staging_table}")
65
66     print(f" {label:<34} {row_count:>9,} rows ({mode_used})")
67     results.append({"entity": label, "target": target_table,
68                   "rows": row_count, "status": "COMMITTED"})

```

[5] ✓ - Command executed in 44 sec 848 ms on 8/9/2026, 11:13:29 PM

FACIL.bed	101 rows	(append)
FACIL.department	49 rows	(append)
FACIL.hospital	5 rows	(append)
PHARM.medication_order	97,511 rows	(append)

```

34     "entities_committed": len(committed),
35     "total_rows": total_rows,
36 })

```

[6] ✓ - Command executed in 8 sec 739 ms on 8/9/2026, 11:13:37 PM

Committed : 4 entities, 97,666 rows

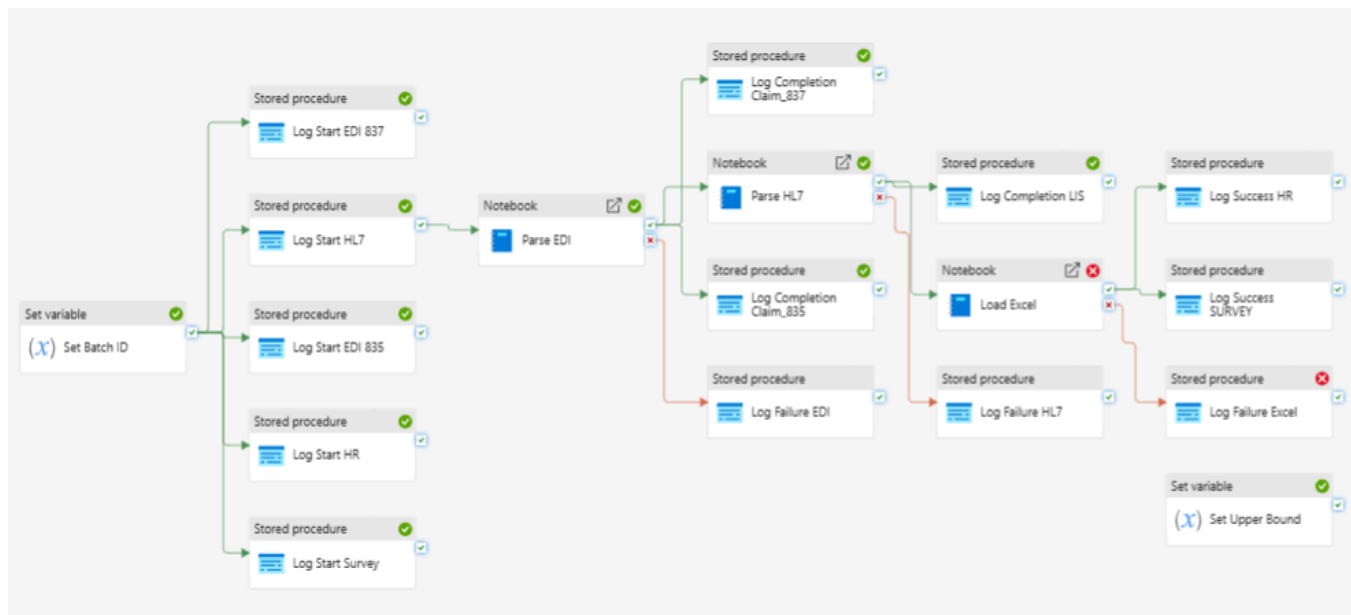
No data : 0 entities

Reconciliation: staging and Bronze row counts match for every entity

ExitValue: {"batch_id": "API_20260810030756_e96a7258", "entities_committed": 4, "total_rows": 97666}

STEP 3: Data Ingestion from FileSource/OneDrive/Sharepoint/Blob

Since my fabric user account doesn't have necessary access and authoz, using onedrive/sharepoint is tricky here, so I drop the CSV files in bzone lakehouse using OneLake Explorer.



> SetBatchID | Set upperBound [setVariable activity to set batchId and watermark value]

Since we have the source data inside the bronze lakehouse, we only have to parse the data with the help of a notebook.

Three notebook have been deployed sequentially 'PARSE EDI', 'PARSE HL7', 'PARSE EXCEL'

Notebook Snapshots

PARSE EDI:

```

1 # This cell is generated from runtime parameters. Learn more: https://go.microsoft.com/fwlink/?linkid=2161015
2 edi_837_path = "Files/landing/CLAIMS/837/*.edi"
3 edi_835_path = "Files/landing/CLAIMS/835/*.edi"
4 load_date = "2026-08-11"
5

```

✓ - Command executed in 291 ms on 8/11/2026, 6:18:04 PM

```

1 from pyspark.sql import functions as F
2 from pyspark.sql.types import (ArrayType, StringType, StructField, StructType)
3
4 batch_id = new_batch_id("CLAIMS", "edi")

```

[1] ✓ - Command executed in 294 ms on 8/11/2026, 6:18:05 PM

```

48 (parsed.write.format("delta").mode("append")
49     .partitionBy("_load_date").option("mergeSchema", "true")
50     .saveAsTable(table))
51 return n_rows
52
53
54 rows_837 = load_edi(edi_837_path, udf_837, "claims_837_service_line", "837P")
55 rows_835 = load_edi(edi_835_path, udf_835, "claims_835_remittance", "835")

```

[6] ✓ - Command executed in 27 sec 664 ms on 8/11/2026, 6:18:33 PM

837P: 60 file(s) -> 3,234 rows (0 with parse errors, 0 with no claim number)
835: 60 file(s) -> 711 rows (0 with parse errors, 0 with no claim number)

```

7         f"({100*orphans/max(1,total_remitted):.2f}%)"
8     if total_remitted and orphans / total_remitted > 0.05:
9         print("WARNING: over 5% orphan remittances – check for missing 837 files "
10              "in the landing zone before trusting the claims reports.")

```

[7] ✓ - Command executed in 10 sec 636 ms on 8/11/2026, 6:18:44 PM

Remittances with no matching submission: 0 / 711 (0.00%)

```

1 # Run accounting belongs to the pipeline. See the note in notebook 11.
2 import json as _json
3 mssparkutils.notebook.exit(_json.dumps({
4     "batch_id": batch_id,
5     "rows_837": rows_837,
6     "rows_835": rows_835,
7     "rows_written": rows_837 + rows_835,
8 })))

```

[8] ✓ - Command executed in 731 ms on 8/11/2026, 6:18:44 PM

ExitValue: {"batch_id": "CLAIMS_edi_20260811T221805", "rows_837": 3234, "rows_835": 711, "rows_written": 3945}

PARSE HL7:

```

1 # This cell is generated from runtime parameters. Learn more: https://go.microsoft.com/fwlink/?linkid=2161015
2 source_path = "Files/landing/LIS/lab"
3 load_date = "2026-08-11"
4

```

[1] ✓ - Command executed in 300 ms on 8/11/2026, 6:19:12 PM

```

1 from pyspark.sql import functions as F
2 from pyspark.sql.types import (ArrayType, StringType, StructField, StructType)
3
4 batch_id = new_batch_id("LIS", "lab_result")

```

1 ✓ - Command executed in 260 ms on 8/11/2026, 6:19:13 PM

```

1 raw = (
2     spark.read.format("binaryFile")
3     .load(f"{source_path}/**/*.hl7")
4     .selectExpr("path AS _source_file", "CAST(content AS STRING) AS value")
5 )
6
7 print(f"HL7 files read: {raw.count()}")

```

[5] ✓ - Command executed in 33 sec 183 ms on 8/11/2026, 6:19:47 PM

HL7 files read: 967

```

15         f"before assuming the source is quiet."
16     )
17     if errors / total > 0.02:
18         raise RuntimeError(
19             f"{100*errors/total:.1f}% of messages failed to parse, above the 2% "
20             f"tolerance. Inspect bronze.lis_lab_result where parse_error is not null
21         )

```

[8] ✓ - Command executed in 24 sec 897 ms on 8/11/2026, 6:20:13 PM

```

Parsed results      : 161,728
with parse errors   : 0 (0.00%)
missing LOINC       : 583 (0.36%)
missing encounter   : 0 (0.00%)

```

```

4  (
5      bronze.write.format("delta")
6      .mode("append")
7      .partitionBy("load_date")
8      .option("mergeSchema", "true")
9      .saveAsTable("lis_lab_result")
10 )
11
12 spark.sql("OPTIMIZE lis_lab_result")

```

[9] ✓ - Command executed in 23 sec 43 ms on 8/11/2026, 6:20:36 PM

```

DataFrame[path: string, metrics:
struct<numFilesAdded:bigint,numFilesRemoved:bigint,numFilesUpdatedWithoutRewrite:bigint,filesAdded:struct<min:bigint,max:bigint,avg:double,totalFiles:bigint,t
otalSize:bigint>,filesRemoved:struct<min:bigint,max:bigint,avg:double,totalFiles:bigint,totalSize:bigint>,filesUpdatedWithoutRewrite:struct<min:bigint,max:big
int,avg:double,totalFiles:bigint,totalSize:bigint>,filesRemovedBreakdown:array<struct<reason:string,metrics:struct<min:bigint,max:bigint,avg:double,totalFiles
:bigint,totalSize:bigint>>,partitionsOptimized:bigint,zOrderStats:struct<strategyName:string,inputCubeFiles:struct<num:bigint,size:bigint>,inputOtherFiles:st
ruct<num:bigint,size:bigint>,inputNumCubes:bigint,mergedFiles:struct<num:bigint,size:bigint>,numOutputCubes:bigint,mergedNumCubes:bigint>,clusteringStats:st
ruct<inputZCubeFiles:struct<numFiles:bigint,size:bigint>,inputOtherFiles:struct<numFiles:bigint,size:bigint>,inputNumZCubes:bigint,mergedFiles:struct<numFiles:b
igint,size:bigint>,numOutputZCubes:bigint>,numBatches:bigint,totalConsideredFiles:bigint,totalFilesSkipped:bigint,preserveInsertionOrder:boolean,numFilesSkipp
edToReduceWriteAmplification:bigint,numBytesSkippedToReduceWriteAmplification:bigint,startTimeMs:bigint,endTimeMs:bigint,totalClusterParallelism:bigint,totals
cheduledTasks:bigint,autoCompactParallelismStats:struct<maxClusterActiveParallelism:bigint,minClusterActiveParallelism:bigint,maxSessionActiveParallelism:bigi
nt,minSessionActiveParallelism:bigint>,deletionVectorStats:struct<numDeletionVectorsRemoved:bigint,numDeletionVectorRowsRemoved:bigint>,numTableColumns:bigint
,numTableColumnsWithStats:bigint>]

```

```

1  # Run accounting belongs to the pipeline, not the notebook. The control
2  # tables live in wh_ohn_control now, and a notebook cannot write to a
3  # Warehouse. The pipeline wraps this notebook in sp_batch_start /
4  # sp_batch_end and reads the counts below out of the exit value.
5  import json as _json
6  mssparkutils.notebook.exit(_json.dumps({
7      "batch_id": batch_id,
8      "files_read": raw.count(),
9      "rows_written": total,
10     "parse_errors": errors,
11     }))

```

✓ - Command executed in 2 sec 8 ms on 8/11/2026, 6:20:38 PM

```
ExitValue: {"batch_id": "LIS_lab_result_20260811T221913", "files_read": 967, "rows_written": 161728, "parse_errors": 0}
```

PARSE EXCEL:

```

1 # CELL 1 - PARAMETERS ← tag this cell
2 batch_id = "FILES_20260811221723_99091347"
3 excel_path = "Files/landing/EXCEL"

```

✓ 3 sec - Command executed in 298 ms by User 1 on 8/11/2026, 6:30:36 PM

>  Log

```

1 # CELL 2
2 from datetime import datetime, timezone
3 import openpyxl
4 from pyspark.sql import functions as F
5
6 run_ts = datetime.now(timezone.utc)
7 load_date = run_ts.date()
8
9 # These workbooks carry a title block in rows 1-3 and headers in row 4,
10 # because that is what human-maintained spreadsheets look like. Reading from
11 # row 1 gives you a header of 'Medical Staff Roster' and one usable column.
12 HEADER_ROW = 4
13
14 FILES = {
15     "doctor_roster.xlsx": ("HR", "doctor", "hr_doctor"),
16     "survey_response_export.xlsx": ("SURVEY", "survey_response", "survey_response"),
17 }

```

✓ <1 sec - Command executed in 748 ms by User 1 on 8/11/2026, 6:30:41 PM

```

26 spark.sql(f"DELETE FROM {target_table} WHERE _load_date = {load_date}")
27
28 (df.write.format("delta").mode("append")
29     .partitionBy("_load_date").option("mergeSchema", "true")
30     .saveAsTable(target_table))
31
32 print(f"{target_table:<24} {df.count():>7,} rows, {len(header)} columns")
33 wb.close()

```

41 ✓ 27 sec - Command executed in 28 sec 44 ms by User 1 on 8/11/2026, 6:31:17 PM

>  Spark jobs (20 of 20 succeeded)  Resources  Log

```

** hr_doctor                240 rows, 14 columns
   survey_response          8,499 rows, 18 columns

```

BRONZE LAKEHOSE TILL NOW AFTER ALL INGESTION:

..

 Table  New chart			
Table view			
	ABC namespace	ABC tableName	0/1 isTemporary
1	`OHN-dev`.lh_bronze.dbo	claims_835_remittance	false
2	`OHN-dev`.lh_bronze.dbo	claims_837_service_line	false
3	`OHN-dev`.lh_bronze.dbo	ehr_admission	false
4	`OHN-dev`.lh_bronze.dbo	ehr_bed_assignment	false
5	`OHN-dev`.lh_bronze.dbo	ehr_diagnosis	false
6	`OHN-dev`.lh_bronze.dbo	ehr_emergency_visit	false
7	`OHN-dev`.lh_bronze.dbo	ehr_patient	false
8	`OHN-dev`.lh_bronze.dbo	facil_bed	false
9	`OHN-dev`.lh_bronze.dbo	facil_department	false
10	`OHN-dev`.lh_bronze.dbo	facil_hospital	false
11	`OHN-dev`.lh_bronze.dbo	fin_invoice	false
12	`OHN-dev`.lh_bronze.dbo	fin_invoice_line	false
13	`OHN-dev`.lh_bronze.dbo	fin_patient_account	false
14	`OHN-dev`.lh_bronze.dbo	hr_doctor	false
15	`OHN-dev`.lh_bronze.dbo	lis_lab_result	false
16	`OHN-dev`.lh_bronze.dbo	pharm_medication_order	false
17	`OHN-dev`.lh_bronze.dbo	ref_code_mapping	false

16	`OHN-dev`.lh_b...	pharm_medicat...	false
17	`OHN-dev`.lh_b...	ref_code_mappi...	false
18	`OHN-dev`.lh_b...	ref_icd10ca	false
19	`OHN-dev`.lh_b...	ref_loinc	false
20	`OHN-dev`.lh_b...	ref_medication	false
21	`OHN-dev`.lh_b...	ref_payer	false
22	`OHN-dev`.lh_b...	sched_appoint...	false
23	`OHN-dev`.lh_b...	sched_appoint...	false
24	`OHN-dev`.lh_b...	sched_patient	false
25	`OHN-dev`.lh_b...	survey_response	false